DocuMind AI

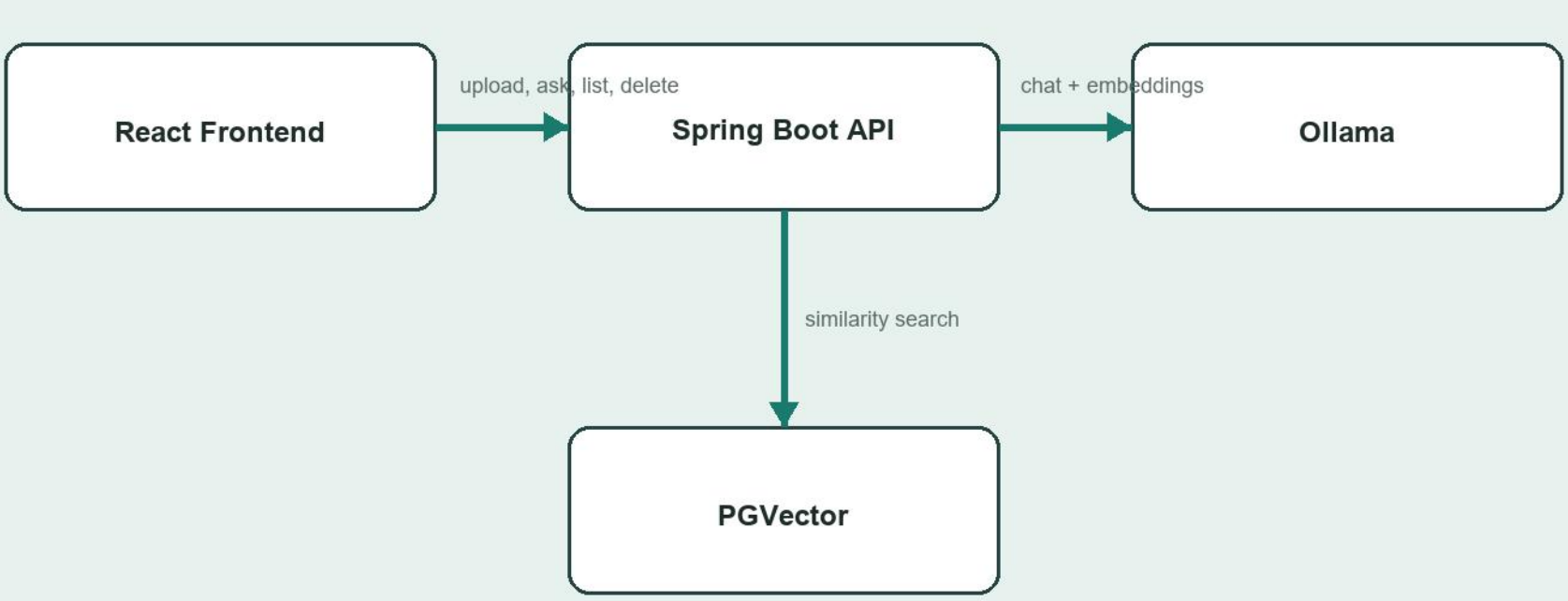
Project Explanation Report

This report explains what we have built so far for DocuMind AI, a RAG-based document question-answering system using Spring Boot, Ollama, PostgreSQL with PGVector, and a React frontend.

Current Capability Snapshot

- Upload PDF documents and extract readable text.
- Split large documents into chunks and embed them safely.
- Store vectors in PGVector and retrieve relevant chunks for questions.
- Answer with RAG using Ollama, including source previews.
- Manage indexed documents through a React interface.
- Scope questions to one chosen PDF or search all indexed documents.

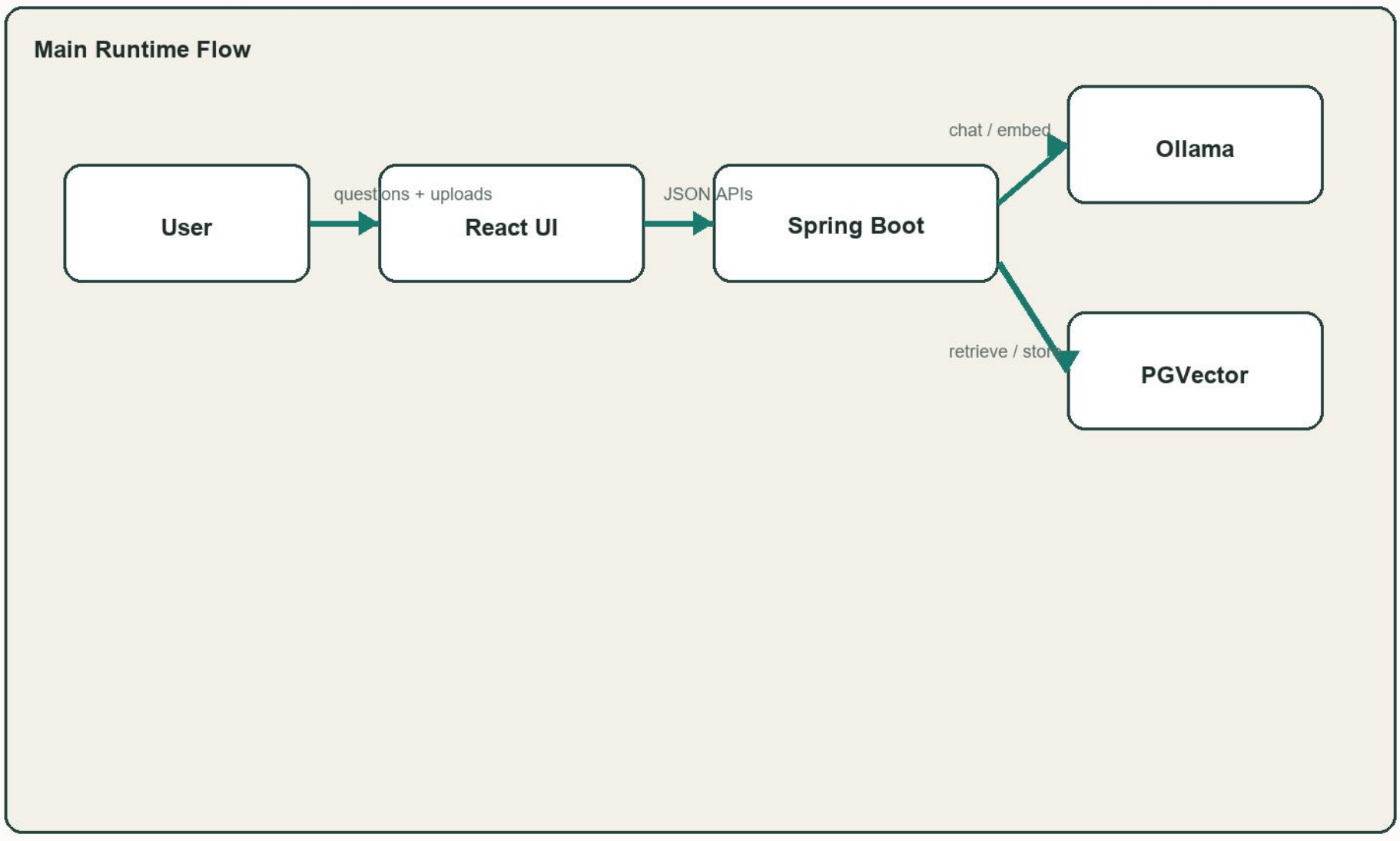
System Overview



Generated in workspace for project handoff and explanation.

Architecture And Data Flow

DocuMind AI is organized around a clean RAG pipeline. The frontend drives the experience, the Spring backend coordinates document ingestion and retrieval, Ollama handles embeddings and answer generation, and PGVector stores searchable vectors.



Endpoints In Use

- POST /documents/upload: parse a PDF, chunk it, embed each chunk, and store vectors.
- GET /documents: return indexed document names with chunk counts.
- DELETE /documents/{filename}: remove all indexed chunks for a chosen PDF.
- GET /ai/ask?message=...: perform similarity search and answer from retrieved context.
- GET /ai/ask?message=...&filename=...: the same flow but restricted to a single document.

What We Built

Backend Foundations

- Spring Boot app with Ollama and PGVector configuration.
- Docker Compose stack for app, Ollama, and PostgreSQL.
- Multistage Dockerfile so source changes rebuild into the container cleanly.

Ingestion Pipeline

- PDF parsing through PDFBox via LangChain4j.
- Chunking added to prevent embedding context overflow.
- Chunk metadata now stores filename and chunk index.

RAG Answering

- Similarity search over PGVector wired into ChatService.
- Grounded prompt built from retrieved chunks instead of plain chat.
- Source previews returned with each answer.

Frontend Experience

- React and Vite interface built in a separate frontend workspace.
- Upload, chat, source preview, and indexed-document panels.
- Dockerized frontend running on port 5173.

Document Management

- List indexed documents with chunk counts.
- Delete document support from both API and UI.
- Automatic refresh after upload or delete.

Search Controls

- Search scope selector for all documents or one chosen PDF.
- Backend metadata filter applied on similarity search.
- Selection stays aligned with current indexed documents.

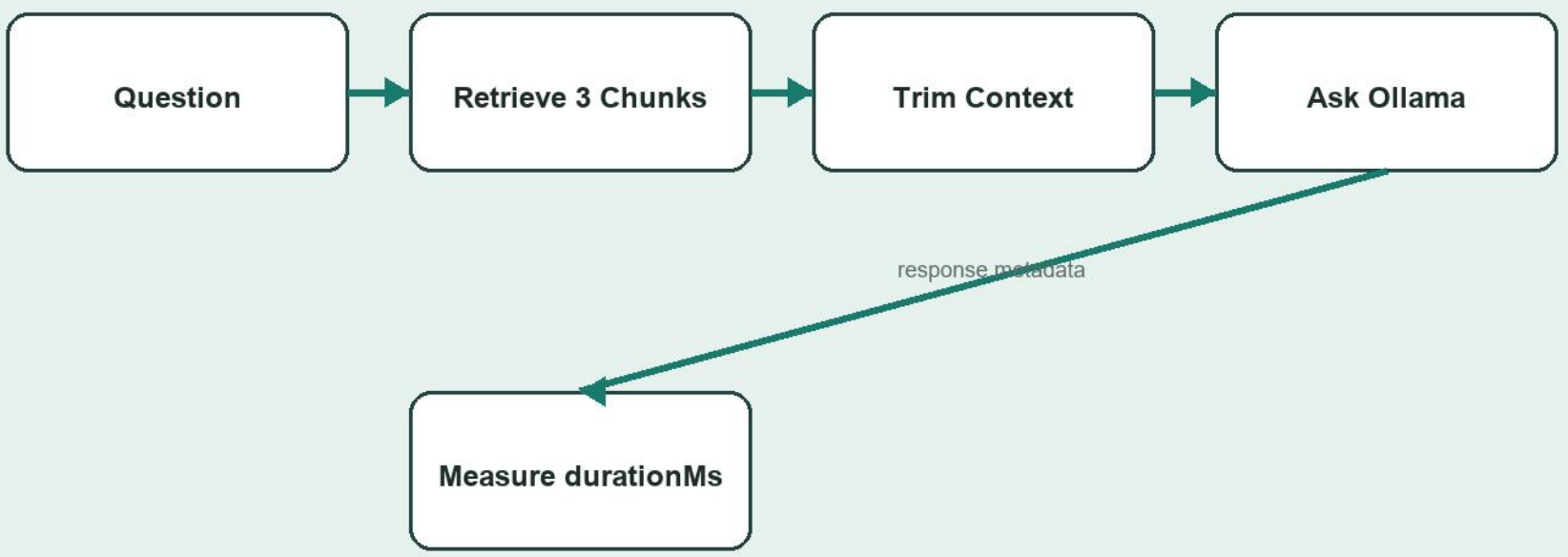
RAG Tuning And Current Direction

After scoped Q and A was working, the main remaining issue became answer speed and answer discipline. A scoped request succeeded, but it took a long time and the model sometimes added extra examples. We started tuning the retrieval and prompt shape to improve that.

Tuning Changes Added

- Lowered retrieved chunk count from 4 to 3.
- Raised similarity threshold slightly to reduce weaker matches.
- Trimmed each chunk context before sending it to the LLM.
- Added a total context cap so prompts stay smaller.
- Tightened the instruction so the model stays grounded in retrieved text.
- Added durationMs and scope to the API response for visibility.
- Frontend now shows response time per assistant message.

Current Improvement Path



Recommended Next Steps

The project is now a usable end-to-end RAG application. The next steps are mostly about polish, answer quality, and product maturity rather than basic plumbing.

Priority Roadmap

- Finish validating the new RAG tuning path after restarting the latest containers.
- Consider a stronger local chat model than tinyllama if hardware allows, because model quality affects grounded answers heavily.
- Add chat history or saved sessions so questions survive page refreshes.
- Store richer metadata such as upload time, page number, and perhaps user tags.
- Add tests for upload, retrieval, and document management endpoints.
- Improve source presentation by showing page-level citations when available.
- Add authentication later if multiple users will manage their own document spaces.

Deliverables In Workspace

Backend code, frontend code, Docker setup, and this explanation PDF all live in the docbot workspace. The frontend runs at http://localhost:5173 and the backend API runs at http://localhost:8080.